COMPUTER NETWORKS ASSESSMENT

Constraint-Based Problem Solving

HTTP – Designing a Web Access Solution

Name	**Ajay Kumar Reddy**
Register Number	**1924252724**
Course / Slot	**CSA 0712 / D_____**
Faculty	**Dr. S. Salini**

1. Problem Statement

A company is developing a small web application named intranet.local. The application must be accessible only inside the company network and must support both static resources such as HTML, CSS and images, and dynamic content generated by a backend server. The HTTP solution must satisfy caching, server identification, default-port and efficient page-loading requirements.

2. Constraints Considered

- The application must be accessible only within the company network.
- Static files must be cached by the browser for 24 hours.
- Dynamic content must not be cached.
- The web server must listen on the default HTTP port 80.
- Every response must identify the server as IntraWeb/1.0.
- The number of client-server round trips should be minimized.

3. Task (a) – HTTP Headers

The following headers and values satisfy the caching, server-identification and round-trip requirements.

Requirement	HTTP Header	Value	Purpose
Static files cached for 24 hours	Cache-Control	public, max-age=86400	Allows browser/proxy caching for 86,400 seconds (24 hours).
Static files cached for 24 hours	Expires	Date/time + 24 hours	Provides an explicit expiration time for HTTP/1.0-compatible caching.
Dynamic content not cached	Cache-Control	no-store, no-cache, must-revalidate	Prevents storing and forces validation of dynamic responses.

Dynamic content not cached	Pragma	no-cache	Provides compatibility with older HTTP/1.0 caches.
Server identification	Server	IntraWeb/1.0	Identifies the web server in every response.
Minimize round trips	Connection	keep-alive	Keeps the TCP connection open so multiple resources can use the same connection.

The web server should listen on TCP port 80, which is the default port for HTTP. Access restriction to the company network can be implemented using the internal DNS record for intranet.local together with firewall/access-control rules that block external access.

4. Task (b) – Sequence of HTTP Requests and Responses

For the first visit, assume the browser has no cached copies of the required resources. The browser requests the main HTML document and then obtains its CSS, JavaScript and image. HTTP persistent connections are used so that the same TCP connection can remain open.

Step 1 – DNS Resolution

The browser resolves intranet.local using the company's internal DNS. The DNS server returns the private/internal IP address of the IntraWeb server. External DNS should not expose the internal application.

Step 2 – TCP Connection

The browser establishes a TCP connection to the internal server on port 80. HTTP/1.1 persistent connection (keep-alive) is used.

Step 3 – Request HTML

Client → Server: GET /dashboard HTTP/1.1, Host: intranet.local. The server returns 200 OK with Server: IntraWeb/1.0. The HTML response contains references to the CSS, JavaScript and image resources.

Step 4 – Request Static Resources

The browser requests /style.css, /script.js and /dashboard.jpg over the persistent connection. The server returns each resource with Cache-Control: public, max-age=86400 and Server: IntraWeb/1.0. These resources can therefore be stored in the browser cache for 24 hours.

Step 5 – Dynamic Content

If dashboard data is generated dynamically, the browser sends the required request to the backend endpoint. The response contains Cache-Control: no-store, no-cache, must-revalidate and Server: IntraWeb/1.0, so dynamic data is not stored in the browser cache.

Step 6 – Connection Reuse

The persistent connection is kept alive while the browser retrieves the required resources. This avoids creating a separate TCP connection for every file and reduces connection setup overhead.

Round-trip optimization: The design uses persistent HTTP connections (keep-alive) and allows the browser to request multiple independent static resources without repeatedly establishing new TCP connections. In an HTTP/2 implementation, the optimization can be improved further by multiplexing the HTML, CSS, JavaScript and image requests over a single TCP connection.

Simplified Request–Response Flow

Stage	Client Request	Server Response
1	GET /dashboard	200 OK + HTML + Server: IntraWeb/1.0
2	GET /style.css	200 OK + CSS + Cache-Control: public, max-age=86400
3	GET /script.js	200 OK + JavaScript + Cache-Control: public, max-age=86400
4	GET /dashboard.jpg	200 OK + Image + Cache-Control: public, max-age=86400
5	GET /api/dashboard-data	200 OK + Dynamic data + Cache-Control: no-store, no-cache, must-revalidate

5. Task (c) – Changes Required for HTTPS

1. Install a valid TLS/SSL certificate for intranet.local on the web server.
2. Configure an HTTPS listener on TCP port 443, the default HTTPS port.
3. Keep the existing HTTP port 80 only if needed for redirection to HTTPS; otherwise restrict or disable it according to company policy.
4. Use HTTPS/TLS for all requests so that credentials, dynamic data and static resources are encrypted in transit.
5. Keep the same caching rules: static resources can use Cache-Control: public, max-age=86400, while dynamic responses should use Cache-Control: no-store, no-cache, must-revalidate.
6. Continue sending Server: IntraWeb/1.0 in all HTTP responses.
7. Configure internal DNS and firewall rules so that the service remains accessible only from the company network.
8. Enable HTTP persistent connections over HTTPS. HTTP/2 can additionally be enabled over TLS to multiplex multiple resource requests and reduce page-load latency.

6. Conclusion

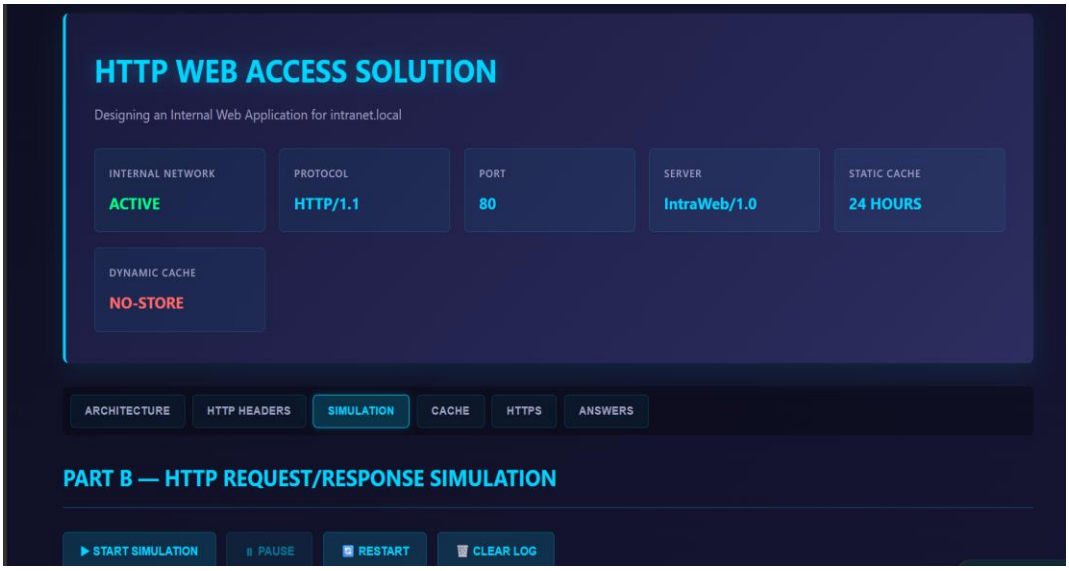
The proposed HTTP solution satisfies the constraints by restricting access to the internal company network, caching static resources for 24 hours, preventing caching of dynamic content, using port 80

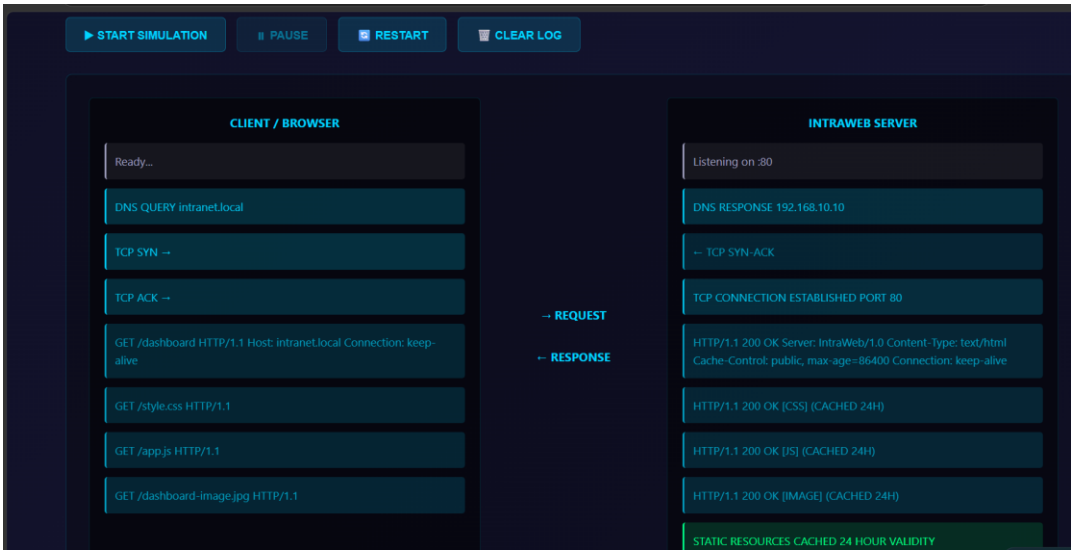
for HTTP, and identifying the server as IntraWeb/1.0. Persistent connections and resource reuse reduce unnecessary connection setup and therefore improve the typical page-load performance. If HTTPS is introduced, TLS certificates, port 443, encrypted communication and appropriate HTTPS server configuration must be added while retaining the same caching and access-control policies.

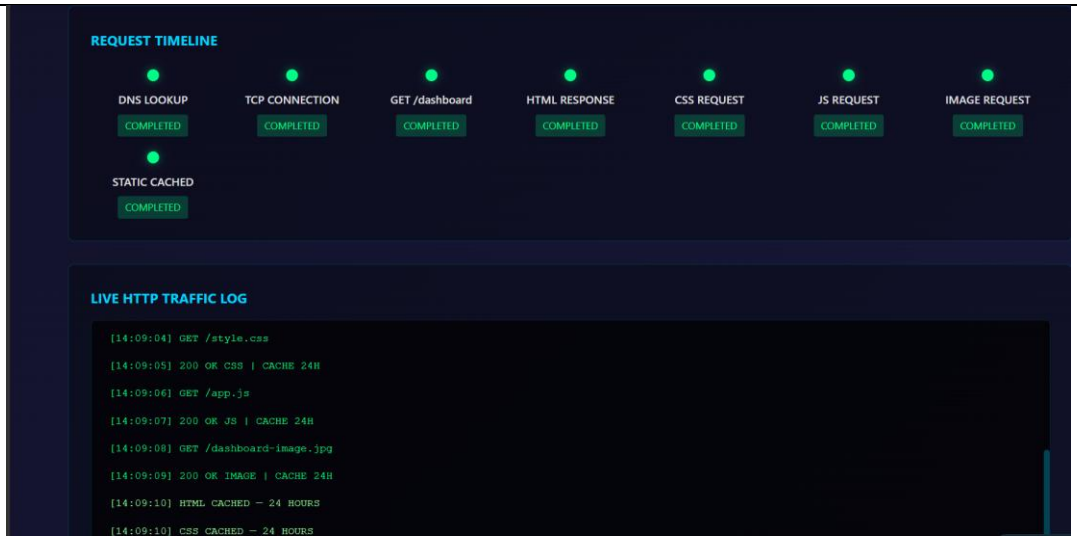
7. Website Implementation – Screenshot

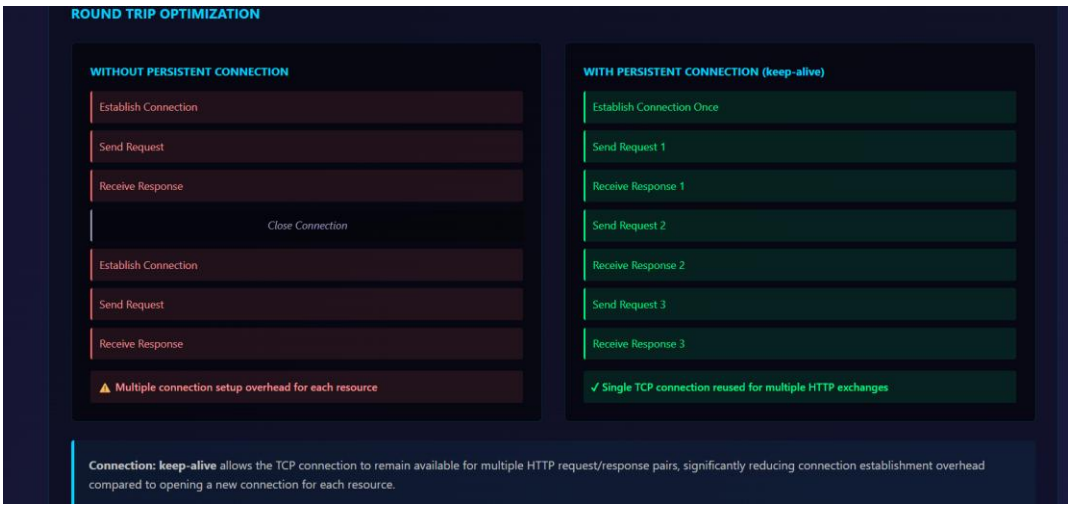
Insert the screenshot of the implemented intranet.local website below. The screenshot should clearly show the working web page/dashboard.

[INSERT WEBSITE SCREENSHOT HERE]









8. Website

Website Link: [HTTP Web Access Solution - intranet.local]